Stats 21: Homework 1

Kayla Tjenalooi

Acknowledgements: several of these problems are copied from or modified from Think Python by Allen Downey.

I've started the homework file for you. You'll need to fill in the rest with your answers.

My encouragement is to use the keyboard shortcuts as much as possible and use the mouse as little as possible while working the Jupyter Notebook.

After you complete the homework with your answers, in the toolbar, select "Clear All Outputs" and then click "Run All".

Review the document to **make sure all requested output is visible**. You will not get credit for problems where the requested output is not visible, even if the function you coded is correct.

When you are satisfied with the output, click the `...` (to the right of Outline), select "Export", and then Export to HTML.Finally, open the HTML file and "Print as PDF". Submit the PDF to Gradescope.

Submit this ipynb file, complete with your answers and output to Canvas / Bruin Learn.

Again, you must make sure all requested output is visible to receive full credit.

Task 1

Create an account on GitHub.

Change your profile picture. Ideally, use photo of yourself that would be appropriate for a resume. If you are not comfortable with the idea of using a photo of yourself, use any other image that is suitable for a workplace environment.

Follow the instructions provided in class to fork the class repository to your GitHub.

Create another repository with at least two text files in it on GitHub (other than the forked class notes repository). Make at least two additional commits to the reposiotry and push them to GitHub.

Provide a link to both repositories here.

You will also need to submit the link to your own repository (not the forked one) to Canvas / Bruin Learn.

Your Answer:

- Link to your forked repository: https://github.com/kaylatjenalooi/26-spring-stats-21
- Link to your own repository: https://github.com/kaylatjenalooi/hw1_repository

Problem 2

An important part of programming is learning to interpret error messages and understanding what correction needs to be made.

Read and familiarize yourself with the following error messages.

Explain the error. Then duplicate each cell and correct the error. The first problem has been done for you as an example.

In [1]:
```
# A
print("Hello World"
```

```
  Cell In[1], line 2
    print("Hello World"
                       ^
SyntaxError: incomplete input
```

Answer: The `print()` function is missing the closing parenthesis. This results in an unexpected EOF error.

In [2]:
```
# corrected:
print("Hello World")
```

```
Hello World
```

In [ ]:
```
# B
print("Hello")
    print("Goodbye")
```

```
  Cell In[3], line 3
    print("Goodbye")
    ^
IndentationError: unexpected indent
```

Answer: There is an unexpected indent in the second line, resulting in an identation error

In [1]:
```
# corrected:

print("Hello")
print("Goodbye")
```

```
Hello
Goodbye
```

In [4]:
```
# C
x = 10
if x > 8
    print("x is greater than 8")
```

```
  Cell In[4], line 3
    if x > 8
           ^
SyntaxError: expected ':'
```

Answer: There is a missing colon after the if-statement, resulting in a syntax error.

In [2]:
```
# corrected
x = 10
if x > 8:
    print("x is greater than 8")
```

```
x is greater than 8
```

In [5]:
```
# D
if x = 10:
    print("x is equal to 10")
```

```
  Cell In[5], line 2
    if x = 10:
       ^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Answer: The comparison operator is '==' not '=', which caused a syntax error.

In [3]:
```
# corrected
if x == 10:
    print("x is equal to 10")
```

```
x is equal to 10
```

In [6]:
```
# E
x = 5
if x == 5:
print("x is five")
```

```
  Cell In[6], line 4
    print("x is five")
    ^
IndentationError: expected an indented block after 'if' statement on line 3
```

Answer: There should be an indent for the line where print is, because there isn't, it causes an identation error.

In [4]:
```
# corrected
x = 5
if x == 5:
    print("x is five")
```

```
x is five
```

In [6]:
```
# F
l = [1, 2, 50, 10]
l = sort(l)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[6], line 3
      1 # F
      2 l = [1, 2, 50, 10]
----> 3 l = sort(l)

NameError: name 'sort' is not defined
```

Answer: The sort function is not defined, so it can't be used to sort the list, resulting in a name error. Sorted should have been used instead.

In [8]:
```
# corrected
l = sorted(l)

# to check
print(l)
```

```
[1, 2, 10, 50]
```

Problem 3

Use Python as a calculator. Enter the appropriate calculation in a cell and be sure the output value is visible.

A. How many seconds are there in 42 minutes 42 seconds?

In [15]:
```
total_seconds = 42 * 60 + 42
print(f'There are {total_seconds} seconds in 42 minutes and 42 seconds.')
```

```
There are 2562 seconds in 42 minutes and 42 seconds.
```

B. There are 1.61 kilometers in a mile. How many miles are there in 10 kilometers?

In [ ]:
```
miles_in_10km = 10 / 1.61
print(f'There are {miles_in_10km} miles in 10 kilometers.')
```

```
There are 6.211180124223602 miles in 10 kilometers
```

C. If you run a 10 kilometer race in 42 minutes 42 seconds, what is your average 1-mile pace (time to complete 1 mile in minutes and seconds)? What is your average speed in miles per hour?

In [19]:
```
#divide seconds by miles in 10km

avg_sec_per_mile = total_seconds/miles_in_10km
```

```
# convert seconds into minutes and seconds
avg_min = avg_sec_per_mile // 60
sec_remainder = avg_sec_per_mile % 60

print(f'My average 1-mile pace would be {avg_min} minutes and {sec_remainder

# convert seconds per mile into miles per hour by multiplying the reciprocal
mph = 1 / avg_sec_per_mile * 3600

print(f'My average speed in miles per hour is {mph}.')
```

```
My average 1-mile pace would be 6.0 minutes and 52.48200000000003 seconds.
My average speed in miles per hour is 8.727653570337615.
```

Problem 4

Write functions for the following problems.

A. The volume of a sphere with radius r is

$$V = \frac{4}{3}\pi r^3$$

Write a function `sphere_volume(r)` that will accept a radius as an argument and return the volume.

- Use the function to find the volume of a sphere with radius 5.
- Use the function to find the volume of a sphere with radius 15.

In [23]:
```
import math

def sphere_volume(r):
    return 4 / 3 * math.pi * r**3

print(sphere_volume(5))
print(sphere_volume(15))
```

```
523.5987755982989
14137.166941154068
```

B. Suppose the cover price of a book is $24.95, but bookstores get a 40% discount. Shipping costs $3 for the first copy and 75 cents for each additional copy.

Write a function `wholesale_cost(books)` that accepts an argument for the number of books and will return the total cost of the books plus shipping.

- Use the function to find the total wholesale cost for 60 copies.
- Use the function to find the total wholesale cost for 10 copies.

In [26]:
```
def wholesale_cost(books):
    if books <= 0:
        return 0
```

```
    cover_price = 24.95 * 0.6 * books
    shipping_price = 3 + (books - 1) * 0.75
    return cover_price + shipping_price

print(wholesale_cost(60))
print(wholesale_cost(10))
```

```
945.4499999999999
159.45
```

C. A person runs several miles. The first and last miles are run at an 'easy' pace. Other than the first and last miles, the other miles are at a faster pace.

Write a function `run_time(miles, warm_pace, fast_pace)` to calculate the time the runner will take. The function accepts three input arguments: how many miles the runner travels (minimum value is 2), the warm-up and cool-down pace, the fast pace. The function will print the time in the format minutes:seconds, and will return a tuple of values: (minutes, seconds)

Use the function to find the time to run a total of 5 miles. The warm-up pace is 8:15 per mile. The speed pace is 7:12 per mile.

Call the function using: `run_time(miles = 5, warm_pace = 495, fast_pace = 432)`

In [32]:
```
def run_time(miles, warm_pace, fast_pace):
    total_time = (warm_pace * 2) + fast_pace * (miles - 2)

    minutes = total_time // 60
    seconds = total_time % 60

    print(f'{minutes}:{seconds}')
    return minutes, seconds

run_time(miles = 5, warm_pace = 495, fast_pace = 432)
```

```
38:6
```

Out[32]:
```
(38, 6)
```

Another important skill is to be able to read documentation.

Read the documentation for the function str.split() at https://docs.python.org/3/library/stdtypes.html#str.split

Adjust the function so that the call can be made with minutes and seconds:

`run_time(miles = 5, warm_pace = "8:15", fast_pace = "7:12")`

In [35]:
```
def run_time (miles, fast_pace, warm_pace):
    fast_min = int(fast_pace.split(':')[0])
    fast_sec = int(fast_pace.split(':')[1])
    warm_min = int(warm_pace.split(':')[0])
```

```
    warm_sec = int(warm_pace.split(':')[1])

    total_time_in_sec = (fast_min * 60 * (miles - 2)) + (fast_sec * (miles -

    minutes = total_time_in_sec // 60
    seconds = total_time_in_sec % 60

    print(f'{minutes}:{seconds}')
    return minutes, seconds

run_time(miles = 5, warm_pace = "8:15", fast_pace = "7:12")
```

```
38:6
```

Out[35]: (38, 6)

Problem 5

Use `import math` to gain access to the math library.

Create a function `polar(real, imaginary)` that will return the polar coordinates of a complex number.

The input arguments are the real and imaginary components of a complex number. The function will return a tuple of values: the value of the radius `r` and the angle `theta`.

For a refresher, see: https://ptolemy.berkeley.edu/eecs20/sidebars/complex/polar.html

Show the results for the following complex numbers:

- 1 + i
- -2 - 3i
- 4 + 2i

In [31]:

```
# math is already imported for pi in problem 4

def polar(real, imaginary):
    r = (real**2 + imaginary**2)**(1/2)

    #accounting for different quadrants
    theta = math.atan2(imaginary, real)
    return r, theta

print(polar(1, 1))
print(polar(-2, -3))
print(polar(4, 2))
```

```
(1.4142135623730951, 0.7853981633974483)
(3.605551275463989, -2.158798930342464)
(4.47213595499958, 0.4636476090008061)
```

Problem 6

Define a function called `insert_into(listname, index, iterable)`. It will accept three arguments, a currently existing list, an index, and another list/tuple that will be inserted at the index position.

Python's built-in function, `list.insert()` can only insert one object.

In [1]:
```
# write your code here

def insert_into(listname, index, iterable):
    listname[index:index] = iterable
    return listname
```

In [2]:
```
# do not modify. We will check this result for grading
l = [0,'a','b','c',4,5,6]
i = ['hello', 'there']
insert_into(l, 3, i)
```

Out[2]:
```
[0, 'a', 'b', 'hello', 'there', 'c', 4, 5, 6]
```

Problem 7

Define a function called `first_equals_last(listname)`

It will accept a list as an argument. It will return `True` if the first and last elements are equal and the if the list has a length greater than 1. It will return False for all other cases.

In [1]:
```
# write your function here

def first_equals_last(listname):
    if (listname[-1] == listname[0]) and (len(listname) > 1):
        return True
    #default
    return False
```

In [2]:
```
# do not modify. We will check this result for grading
a = [1,2,3]
first_equals_last(a)
```

Out[2]:
```
False
```

In [3]:
```
# do not modify. We will check this result for grading
b = ['hello','goodbye','hello']
first_equals_last(b)
```

Out[3]:
```
True
```

In [4]:
```
# do not modify. We will check this result for grading
c  = [1,2,3,'1']
first_equals_last(c)
```

Out[4]: False

In [5]:
```
# do not modify. We will check this result for grading
d = [[1,2],[3,2],[1,2]]
first_equals_last(d)
```

Out[5]: True